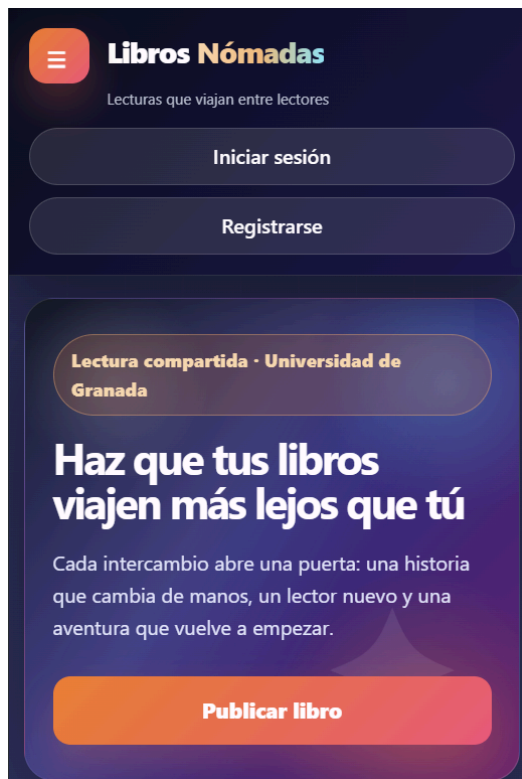


Memoria del desarrollo del proyecto

Libros Nómadas

Proyecto final de Tecnologías Web

Aplicación	Libros Nómadas
Tecnologías principales	Laravel, PHP, MySQL, Blade, Bootstrap 5, HTML5 y CSS
Tipo de proyecto	Plataforma para intercambio de libros entre usuarios



Índice

1. Descripción general del proyecto
2. Tecnologías utilizadas y estructura Laravel
3. Diseño de la base de datos mediante migraciones
4. Modelos Eloquent y relaciones principales
5. Gestión de usuarios, sesiones y roles
6. Catálogo, búsqueda y detalle de libros
7. Publicación de libros y subida de imágenes
8. Solicitudes de intercambio
9. Perfil de usuario e historial
10. Sistema de disputas y administración
11. Contacto, Bootstrap, diseño visual y responsive
12. Validación, seguridad y pruebas
13. Conclusión
14. Enlace a la carpeta compartida

1. Descripción general del proyecto

En este proyecto hemos desarrollado la propuesta de web 3.7 (intercambio de libros de segunda mano). La idea principal ha sido crear una plataforma donde los usuarios puedan publicar libros que ya no utilizan e intercambiarlos con otros usuarios, fomentando de este modo la economía circular.

La aplicación se llama Libros Nómadas porque la idea es que cada libro pueda viajar entre lectores y no quedarse parado. Con esto buscamos simular un servicio sencillo de intercambio de libros de segunda mano, fomentando la lectura y dando una nueva vida a ejemplares que normalmente quedarían olvidados en una estantería.

La web cuenta con una interfaz de acceso público, desde la que cualquier visitante puede consultar el catálogo y buscar libros, y una parte privada para usuarios registrados. Una vez iniciada la sesión, el usuario puede publicar libros, solicitar intercambios, gestionar sus solicitudes y consultar su perfil. También hemos incorporado un rol de administrador, pensado para gestionar categorías y revisar disputas relacionadas con intercambios ya aceptados.

En la primera versión de la práctica desarrollamos la aplicación con PHP, MySQL, HTML5 y CSS. Posteriormente, para dejar el proyecto más completo y profesional, adaptamos la estructura a Laravel e incorporamos Bootstrap 5. La lógica se reorganizó en rutas, controladores, modelos, migraciones y vistas Blade, manteniendo la misma idea funcional del proyecto original pero con una arquitectura más ordenada y cercana al modelo MVC.

2. Tecnologías utilizadas y estructura Laravel

La versión final se organiza como una aplicación Laravel ejecutada en local mediante `php artisan serve`. La conexión con la base de datos se configura en el archivo `.env` y el acceso a los datos se realiza mediante modelos Eloquent. La parte visual se mantiene en una hoja de estilos propia y se complementa con Bootstrap 5 para formularios, botones, tablas y componentes responsive.

Elemento	Uso dentro del proyecto
Laravel	Framework principal. Organiza la aplicación en rutas, controladores, modelos, migraciones y vistas Blade.
PHP	Lenguaje utilizado por Laravel para procesar formularios, controlar sesiones, validar datos y aplicar la lógica de negocio.
MySQL	Almacena usuarios, libros, categorías, solicitudes de intercambio, disputas y sesiones.
Eloquent ORM	Permite trabajar con las tablas mediante modelos como User, Libro, Categoría, Solicitud y Disputa.
Blade	Sistema de plantillas de Laravel usado para construir las vistas del catálogo, perfil, login, registro, libros, administración y contacto.
Bootstrap 5	Se incorpora mediante CDN para reforzar formularios, botones, tablas y adaptación responsive.
CSS propio	Centraliza el tema oscuro, las tarjetas, el hero, el menú lateral, el menú desplegable y los ajustes responsive.
Composer y Artisan	Composer gestiona dependencias y Artisan se usa para crear controladores, migraciones, seeders y ejecutar el servidor local.

Entre los archivos principales se encuentran:

- **routes/web.php:** Define las rutas públicas, privadas y de administración de la aplicación.

- **app/Http/Controllers:** Contiene los controladores AuthController, CatalogoController, LibroController, PerfilController, SolicitudController, DisputaController, CategoriaController y ContactoController.
- **app/Models:** Incluye los modelos User, Categoria, Libro, Solicitud y Disputa, con sus relaciones principales.
- **database/migrations:** Contiene la creación de las tablas usuarios, categorías, libros, solicitudes y disputas.
- **database/seeders/DatosPruebaSeeder.php:** Inserta usuarios, categorías y libros iniciales para poder probar la aplicación sin introducir datos manualmente.
- **resources/views:** Contiene las vistas Blade, separadas en carpetas para catálogo, autenticación, perfil, libros, administración y contacto.
- **resources/views/layouts/app.blade.php:** Sustituye a header.php y footer.php, sirviendo como plantilla común para todas las páginas.
- **public/css/estilos.css:** Hoja de estilos con tema oscuro, Bootstrap adaptado y diseño responsive.
- **public/uploads:** carpeta donde se guardan y se sirven las portadas de los libros.

3. Diseño de la base de datos mediante migraciones

La información de la aplicación se almacena en una base de datos relacional. Hemos usado migraciones para crear la base de datos. Esto permite reconstruir la estructura de tablas con `php artisan migrate:fresh` y cargar datos de prueba con `php artisan migrate:fresh --seed`.

Tabla	Contenido principal	Finalidad
usuarios	id, nombre, email, password, tipo_usuario, timestamps	Identificar a las personas registradas y distinguir entre usuario normal y administrador.
categorias	id, nombre, timestamps	Clasificar los libros para facilitar la navegación y la búsqueda.
libros	id, titulo, autor, descripcion, imagen, id_categoria, id_usuario, estado, timestamps	Guardar los libros publicados, su propietario, su portada y su disponibilidad.
solicitudes	id_libro, id_libro_ofrecido, id_solicitante, id_propietario, mensaje, estado, timestamps	Representar la petición de intercambio entre dos usuarios.
disputas	id_solicitud, id_usuario_reporta, motivo, estado, respuesta_admin, fecha_resolucion, timestamps	Registrar incidencias sobre intercambios aceptados y su resolución por parte del administrador.
sessions	id, user_id, ip_address, user_agent, payload, last_activity	Guardar las sesiones cuando Laravel trabaja con el driver de base de datos.

En la tabla libros decidimos no guardar la imagen como dato binario. En su lugar, almacenamos la ruta dentro de `public/uploads`. De esta forma, la base de datos queda más limpia y Laravel puede servir las portadas como archivos públicos normales.

```
'imagen' => 'uploads/loqueencontrebaajoelsofa.jpg'
```

También utilizamos estados para controlar el flujo de la aplicación. Un libro puede estar disponible o intercambiado, una solicitud puede estar pendiente, aceptada o rechazada, y una disputa puede estar pendiente o resuelta. En Laravel estos estados se definen en las migraciones mediante campos enum y se consultan desde los controladores para decidir qué debe mostrarse al usuario.

Además, añadimos el seeder DatosPruebaSeeder. En él se insertan categorías, usuarios de prueba y varios libros con portadas ya almacenadas en public/uploads. Esto permite comprobar rápidamente el catálogo, las solicitudes y la administración sin tener que crear todos los datos a mano cada vez que se reinicia la base de datos.

4. Modelos Eloquent y relaciones principales

Se usan modelos Eloquent. Cada modelo representa una tabla y define las relaciones necesarias para que Laravel pueda cargar información relacionada de forma más clara.

Modelo	Tabla asociada	Relaciones principales
User	usuarios	hasMany Libro, hasMany Solicitud como solicitante y hasMany Solicitud como propietario.
Categoria	categorias	hasMany Libro.
Libro	libros	belongsTo Categoria y belongsTo User.
Solicitud	solicitudes	belongsTo Libro como libroPedido y libroOfrecido; belongsTo User como solicitante y propietario.
Disputa	disputas	belongsTo Solicitud y belongsTo User como usuarioReporta.

Estas relaciones se aprovechan especialmente en el perfil y en la zona de administración. Por ejemplo, al mostrar una disputa, el controlador puede cargar el usuario que la reportó, la solicitud asociada, el libro pedido, el libro ofrecido, el propietario y el solicitante usando with().

5. Gestión de usuarios, sesiones y roles

La autenticación se gestiona desde AuthController. Este controlador muestra las vistas de login y registro, valida los datos de entrada, crea usuarios nuevos y permite cerrar sesión. Al registrar un usuario, Laravel valida nombre, correo, contraseña y confirmación de contraseña.

```
'email' => ['required', 'email', 'unique:usuarios,email']
'password' => ['required', 'min:6', 'confirmed']
```

Para no almacenar contraseñas en texto plano, usamos Hash::make al registrar y Auth::attempt al iniciar sesión. Cuando el acceso es correcto, regeneramos la sesión con session()->regenerate(), reduciendo riesgos asociados a reutilización de sesión.

```
if (Auth::attempt($datos)) {
    $request->session()->regenerate();
    return redirect()->route('catalogo.index');
}
```

El cierre de sesión también se realiza desde el controlador. En este caso se llama a Auth::logout(), se invalida la sesión y se regenera el token. Además, a diferencia de la primera versión, el logout se envía mediante POST y con token CSRF, evitando cerrar sesión con un enlace simple de tipo GET.

Tipo de usuario	Permisos principales
Visitante	Puede consultar el catálogo, buscar libros, ver detalles básicos, iniciar sesión, registrarse y contactar.
Usuario registrado	Puede publicar libros, solicitar intercambios, consultar su perfil, aceptar o rechazar solicitudes y reportar disputas.

Las rutas privadas se protegen con middleware auth. Para el administrador, además del middleware, los controladores `CategoriaController` y `DisputaController` comprueban que `tipo_usuario` sea `admin` antes de permitir el acceso.

6. Catálogo, búsqueda y detalle de libros

La página principal se genera desde `CatalogoController` y la vista `resources/views/catalogo/index.blade.php`. En ella se muestran únicamente los libros con estado `disponible`. Cada libro aparece en formato de tarjeta con portada, título, autor y botón de ver detalles.

El buscador permite filtrar por título, autor o categoría. Para conseguirlo, el controlador construye una consulta Eloquent sobre el modelo `Libro` y utiliza `whereHas` para buscar también en la relación `categoría`. Así el usuario no tiene que escribir el texto completo ni exacto; basta con introducir parte del título, del autor o el nombre de una categoría.

```
Libro::with('categoria')
    ->where('estado', 'disponible')
    ->when($buscar !== '', function ($query) use ($buscar) {
        $query->where(function ($q) use ($buscar) {
            $q->where('titulo', 'like', "%$buscar%")
            ->orWhere('autor', 'like', "%$buscar%")
            ->orWhereHas('categoria', function ($cat) use ($buscar) {
                $cat->where('nombre', 'like', "%$buscar%");
            });
        });
    });
```

Las categorías también se muestran como chips visuales. Al pulsar una categoría, se envía su nombre como parámetro de búsqueda, reutilizando la misma ruta del catálogo.

El detalle del libro se gestiona en `LibroController` con el método `show()` y la vista `libros/show.blade.php`. En esta página se muestra la portada grande, título, autor, categoría, usuario que lo publicó, estado y descripción. Si el visitante no ha iniciado sesión, se muestra un aviso para iniciar sesión. Si el libro pertenece al usuario actual, aparece un enlace para gestionarlo. Si pertenece a otro usuario y está disponible, se muestra el formulario de solicitud de intercambio.

7. Publicación de libros y subida de imágenes

Los usuarios registrados pueden publicar nuevos libros desde la ruta `/publicar`. Esta ruta llama al método `create()` de `LibroController`, que carga todas las categorías de la base de datos y las envía a la vista `libros/create.blade.php`.

El formulario solicita título, autor, descripción, categoría e imagen de portada. En la vista usamos clases de Bootstrap como `form-control`, `form-select`, `form-label`, `btn` y `w-100`, pero el aspecto final se adapta al tema oscuro mediante `public/css/estilos.css`.

La inserción se realiza en el método `store()` de `LibroController`. Antes de crear el libro, Laravel valida que los campos existan, que la categoría esté registrada en la tabla `categorias` y que la imagen sea realmente una imagen con extensión permitida.

```
'imagen' => ['required', 'image', 'mimes:jpg,jpeg,png,webp']
```

Para evitar colisiones entre imágenes, generamos un nombre único con `uniqid`. Después movemos el archivo a `public/uploads` y guardamos en la tabla `libros` únicamente la ruta relativa.

```
$nombreImagen = uniqid('libro_', true) . '.' . $extension;
$request->file('imagen')->move(public_path('uploads'), $nombreImagen);
```

El propietario puede acceder a sus propios libros desde el perfil. Si abre uno de sus libros, se carga la vista `libros/owner.blade.php`. Desde ahí puede eliminarlo mediante una petición DELETE con `@method("DELETE")` y una casilla de confirmación obligatoria. Además, si la imagen existe físicamente en `public/uploads`, se elimina también del sistema de archivos.

8. Solicitudes de intercambio

El intercambio entre usuarios se gestiona mediante `SolicitudController`. Cuando un usuario quiere pedir un libro, debe elegir uno de sus libros disponibles para ofrecerlo a cambio y escribir un mensaje para el propietario.

En `store()` comprobamos que el libro solicitado exista y esté disponible, que el libro ofrecido pertenezca al usuario actual y que también esté disponible. También se impide solicitar un libro propio, ofrecer el mismo libro que se solicita y crear solicitudes pendientes duplicadas para el mismo libro.

```
if ($libroSolicitado->id_usuario === Auth::id()) {
    return redirect()->route('libros.show', $libroSolicitado)
        ->with('error', 'No puedes solicitar tu propio libro.');
```

Si todo es correcto, se crea una solicitud con estado pendiente. En el perfil del usuario aparece dentro de Solicitudes enviadas, mientras que el propietario del libro la ve en Solicitudes recibidas.

Aceptar o rechazar una solicitud se hace mediante el método `update()` de `SolicitudController`. Si se rechaza, simplemente cambia a estado rechazada. Si se acepta, se utiliza una transacción para actualizar la solicitud y los dos libros implicados de forma conjunta. De este modo, si se produce un error, no queda un intercambio aplicado a medias.

Cuando se acepta el intercambio, los dos libros pasan a estado intercambiado y también se actualiza el propietario de cada uno: el solicitante recibe el libro solicitado y el propietario recibe el libro ofrecido. Además, se rechazan automáticamente otras solicitudes pendientes relacionadas con cualquiera de esos dos libros para evitar conflictos posteriores.

9. Perfil de usuario e historial

El perfil se gestiona desde `PerfilController` y la vista `perfil/index.blade.php`. Solo puede acceder un usuario autenticado, ya que la ruta `/perfil` está protegida con middleware `auth`.

En esta página se muestra una zona privada que actúa como panel de control del usuario. El controlador carga sus datos, sus libros disponibles, el historial de libros intercambiados, las solicitudes enviadas, las solicitudes recibidas y las disputas reportadas. Para hacerlo, se usan consultas Eloquent con relaciones como `libroPedido`, `libroOfrecido`, `solicitante`, `propietario` y `solicitud`.

- **Mis libros en el catálogo:** lista los ejemplares publicados por el usuario y que siguen disponibles.
- **Solicitudes enviadas:** permite consultar qué libros ha pedido el usuario, qué ha ofrecido a cambio y el estado actual.
- **Solicitudes recibidas:** permite aceptar o rechazar peticiones de otros usuarios.
- **Mis disputas reportadas:** muestra el estado de las incidencias abiertas y la respuesta del administrador si ya existe.
- **Historial de intercambios:** muestra los libros que ya han formado parte de un intercambio.

También se prepara un listado de disputas por solicitud para saber si sobre una solicitud aceptada ya existe una incidencia. Esto evita mostrar dos veces la posibilidad de reportar la misma disputa y mantiene la información más clara para el usuario.

10. Sistema de disputas y administración

El sistema de disputas se organiza en DisputaController. No permitimos abrir una disputa sobre cualquier solicitud: primero comprobamos que la solicitud esté aceptada y que el usuario que reporta forme parte del intercambio.

```
if ($solicitud->estado !== 'aceptada') {  
    return redirect()->route('perfil')  
        ->with('error', 'Solo se pueden reportar disputas sobre intercambios aceptados.');
```

También comprobamos si ya existe una disputa asociada a la misma solicitud. De esta manera evitamos duplicar incidencias sobre un mismo intercambio y mantenemos un seguimiento más ordenado.

```
Disputa::where('id_solicitud', $solicitud->id)->exists();
```

El administrador dispone de la ruta /admin/disputas y de la vista admin/disputas.blade.php. Desde ahí puede ver las disputas ordenadas, consultar el usuario que reporta, el libro pedido, el libro ofrecido, el propietario, el solicitante, el motivo y la fecha. También se mantienen enlaces mailto para contactar con las personas implicadas.

La resolución se hace mediante una petición PATCH a admin.disputas.resolve. El administrador escribe una respuesta y marca la disputa como resuelta. En ese momento se actualiza el estado, la respuesta del administrador y la fecha de resolución.

```
'estado' => 'resuelta'  
'respuesta_admin' => $datos['respuesta_admin']  
'fecha_resolucion' => now()
```

El panel de categorías también está restringido al administrador. En CategoriaController se comprueba el rol admin antes de listar, crear o eliminar categorías. La vista admin/categorias.blade.php permite añadir una nueva categoría y borrar las existentes mediante formularios con @csrf y método DELETE.

11. Contacto, Bootstrap, diseño visual y responsive

La estructura común de la página se encuentra en resources/views/layouts/app.blade.php. Este archivo incluye la marca Libros Nómadas, el subtítulo Lecturas que viajan entre lectores, el saludo del usuario cuando ha iniciado sesión, el menú lateral, el menú desplegable móvil, la zona central de contenido y el pie de página.

Bootstrap 5 se incorpora mediante CDN en el layout principal. Lo usamos sobre todo en formularios, botones y tablas, con clases como form-control, form-select, form-label, btn, btn-primary, table, table-dark y w-100. Aun así, el estilo visual principal sigue estando en public/css/estilos.css, donde adaptamos Bootstrap al tema oscuro del proyecto.

En escritorio utilizamos un menú lateral con las opciones principales de navegación. En pantallas pequeñas ocultamos ese menú lateral y mostramos un desplegable implementado sin JavaScript, usando details y summary. Las opciones cambian según el tipo de usuario: visitante, usuario registrado o administrador.

La página contacto.blade.php incluye un formulario HTML5 con nombre, apellidos, correo electrónico y mensaje. El correo se valida con type=email y los campos obligatorios se marcan con required. En el controlador también se valida el formulario antes de mostrar el mensaje de envío correcto.

En cuanto al diseño visual, usamos un tema oscuro con degradados, tarjetas, sombras, botones llamativos y una portada principal con el mensaje "Haz que tus libros viajen más lejos que tú". El responsive se trabaja con media queries: en móvil el hero se reorganiza, las tarjetas pasan a una columna, los formularios ocupan todo el ancho y las tablas pueden desplazarse horizontalmente si es necesario.

12. Validación, seguridad y pruebas

Validación

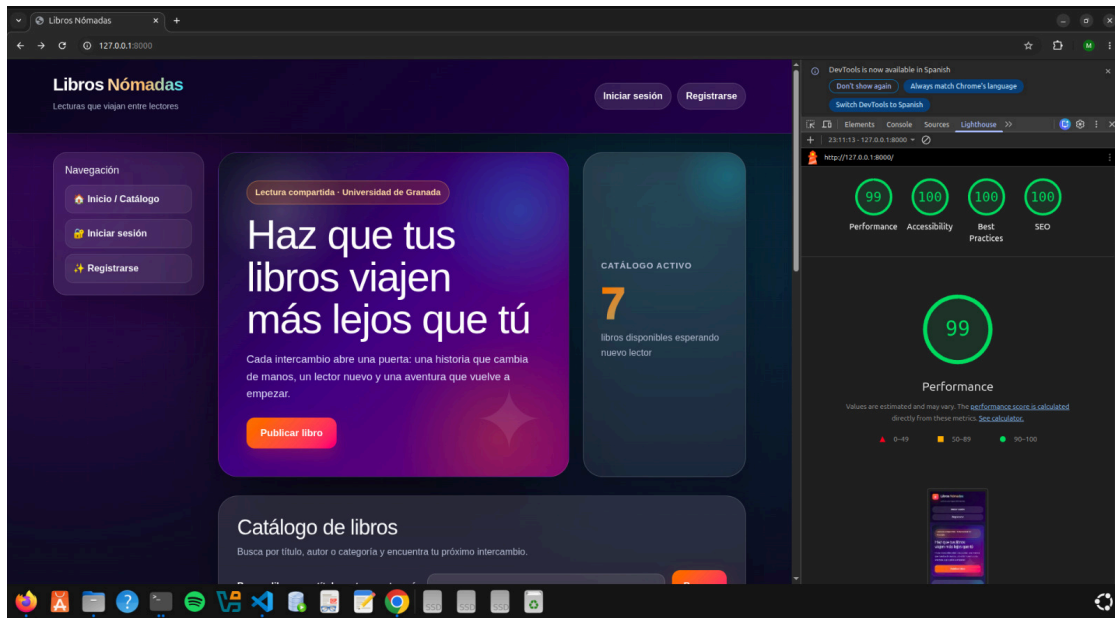
Realizamos la validación en dos niveles. En el navegador usamos atributos HTML5 como `required`, `minlength`, `accept` y `type=email`. En el servidor, Laravel repite las comprobaciones mediante `validate()`, porque no conviene confiar únicamente en la validación del cliente.

Seguridad básica

- Uso de middleware `auth` para proteger las páginas privadas.
- Control de roles en los controladores de administración para permitir solo usuarios `admin`.
- Contraseñas almacenadas con `Hash::make`, no como texto plano.
- Uso de `Auth::attempt` para iniciar sesión de forma integrada con Laravel.
- Uso de `@csrf` en formularios `POST`, `PATCH` y `DELETE`.
- Uso de `@method` para simular `PATCH` y `DELETE` en formularios `HTML`.
- Escapado automático de salida en Blade mediante `{{ }}`.
- Validación de imágenes mediante `image` y mimes: `jpg`, `jpeg`, `png`, `webp`.
- Uso de transacciones al aceptar intercambios para mantener la coherencia de los cambios realizados.

Pruebas realizadas

1. Ejecución de migraciones con `php artisan migrate:fresh`.
2. Carga de datos iniciales con `php artisan migrate:fresh --seed`.
3. Registro de usuario nuevo e inicio automático de sesión.
4. Inicio y cierre de sesión con usuarios normales y administrador.
5. Publicación de libros con portada y categoría.
6. Búsqueda por título, autor y categoría.
7. Visualización del detalle de libros propios y ajenos.
8. Solicitud de intercambio ofreciendo un libro propio.
9. Aceptación y rechazo de solicitudes recibidas.
10. Cambio de estado y propietario de los libros tras aceptar un intercambio.
11. Apertura de disputas solo sobre intercambios aceptados.
12. Prevención de disputas duplicadas.
13. Resolución de disputas desde el panel de administración.
14. Alta y borrado de categorías desde la zona de administración.
15. Revisión del diseño en escritorio, tablet y móvil mediante herramientas del navegador.
16. Prueba con `WAVE` y `Lighthouse` para comprobar accesibilidad y buenas prácticas.



13. Conclusión

Con este proyecto hemos conseguido construir una aplicación web completa alrededor de una idea sencilla: permitir que los usuarios intercambien libros entre sí. A partir de esa idea, hemos desarrollado un catálogo público, un sistema de registro e inicio de sesión, publicación de libros, solicitudes de intercambio, perfil privado, historial, disputas y una zona de administración.

Organizar el proyecto con laravel nos ha permitido trabajar de forma más ordenada, separando rutas, controladores, modelos, migraciones y vistas Blade. Además, la incorporación de Bootstrap 5 ha mejorado la presentación de formularios, botones y tablas, manteniendo al mismo tiempo una identidad visual propia mediante la hoja de estilos personalizada.

Durante el desarrollo hemos puesto en práctica varios contenidos propios de la asignatura: formularios HTML, validación de datos, sesiones, roles, consultas a base de datos, subida de archivos, organización de una aplicación web, separación entre estructura, lógica y diseño, uso de migraciones y control de permisos. También hemos trabajado la parte visual para que la web no fuese solo funcional, sino que tuviera una identidad propia con el nombre Libros Nómadas, tarjetas de libros, tema oscuro, menú lateral, menú desplegable y adaptación responsive.

En definitiva, esta memoria recoge cómo hemos realizado el proyecto y las decisiones principales que hemos tomado para que la aplicación funcione de forma coherente. El trabajo nos ha servido para integrar en una misma práctica base de datos, lógica de servidor, autenticación, formularios, administración e interfaz visual, cumpliendo al mismo tiempo con los requisitos del proyecto.

14. Enlace a la carpeta compartida

<https://github.com/manucazorla69/libros-nomadas>